Music Listening Habits During Recessions

Alexander Tuosto

Research Question

How do signals of poor economic conditions affect music listening habits across genres?

Variables

Play Share by Genre

Total Plays

Last.FM API

Recession Indicator

- Binary Variable 1=Recession, 0=No Recession
- NBER, FRED (USREC)

Unemployment Rate

- Continuous Variable, Unemployed Relative to Labor Force
- BLS, FRED (UNRATE)

PCE Price Index

- Continuous Variable, % change = inflation
- BEA, FRED (PCEPI)

UMich Consumer Sentiment Survey

- Continuous Variable, Index of Consumer Sentiment
- University of Michigan, FRED (UMCSENT)

FRED Variables

```
#_____________________________________________________________________________________________
#BLOCK 1.7 - Getting Fred Data
#In order to make a Fred Dataframe I need to first get the Data, Thankfully this Package allows me to do just that
# pip install fredapi # this entire thing was shown by this https://pypi.org/project/fredapi/
from fredapi import Fred
# gets Fred Package
FREDKEY= '2432515d4bc3acaf9cc6bdbbc8ad8558'
#gets my fred API key
fred=Fred(api_key=FREDKEY)
#structures the variable to use my API key when getting data, this was in the documentation
recession=fred.get_series('USREC')
#recession dummy variable
PCEPI=fred.get_series('PCEPI')
# PCE variable
UNEMP=fred.get_series('UNRATE')
#U3 Unemployment rate
UMICH=fred.get_series('UMCSENT')
#UMICH Consumer sentiment Survey
#_____________________________________________________________________________________________
#BLOCK 1.7.5 - Making Fred DataFrame to Merge
#In order to Merge the Fred Data I first need to make another Dataframe with Fred Data in it
freddf=pd.DataFrame(PCEPI)
freddf['REC']=pd.DataFrame(recession)
#MAKES A COLUMN FOR EACH VARIABLE, this one is titled REC
freddf['UNRATE']=pd.DataFrame(UNEMP)
#Takes the FRED indicator with the tag 'UNEMP' and puts it in a Pandas Dataframe, measures Unemployment
freddf['PCE']=pd.DataFrame(PCEPI)
#Takes the FRED indicator with the tag 'PCE' and puts it in a Pandas Dataframe, measures change in price level
freddf['UMICH']=pd.DataFrame(UMICH)
#Takes the FRED indicator with the tag 'UMICH' and puts it in a Pandas Dataframe, is a survey measuring Consumer Sentiment
freddf.drop(freddf.columns[0],axis=1)
#Drops the first initial useless column
freddf['Year']=freddf.index
#Makes a column from the year, given by the index but just so i can match it with freds year data
freddf['Month']=pd.to_datetime(freddf['Year'], unit='ns').dt.month
#again makes the Month a number so I can sort with it later
freddf['Year']=pd.to_datetime(freddf['Year'], unit='ns').dt.year
#again makes the Year a number so I can sort with it later
#The following link was used for the dt.month and dt.year changes, https://pandas.pydata.org/docs/reference/api/pandas.Series.fillna.html#:~:
```

Last.FM

Music application that pairs with streaming services to track personal listening patterns and recommend new music while also acting as a library for listening data.

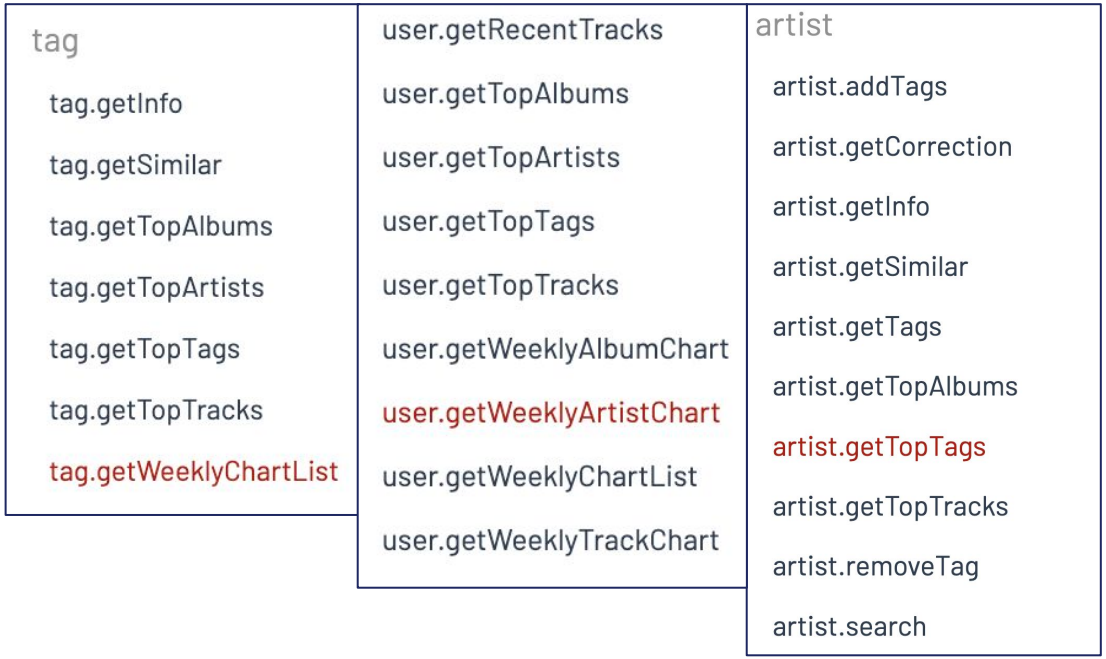
API Documentation

Confusing encoding…

Remedied by consulting their forum and attending office hours

… then straightforward

Each function had a description and an endpoint



API Data Collection

tag.getWeeklyChartList → **user.getWeeklyArtistChart** → **artist.getTopTags**

Collected week data (How many weeks am I looking at?)

Collected artist names for every given week.

Got the top tags for each artist

API Data Collection

Collect Week Data

```
#_____________________________________________________________________________________________
#BLOCK 1.1 - Prime Data, find how many weeks and how to grab tags
#This below block gives me both the tag grabber for each of the artists I'm looking at as well as the weeks I'm counting. This is important
#because I'm looking to do a for loop in a for loop, essentially the main loop is going to cycle weeks and grab the listening data for that
#week, the second loop is going to cycle between either the top 250 artists or the length of the amount of artists, whichever is smaller
#in order to get the weeks I need 'charts', in order to get the top tag and classify each artist I need 'toptagget', cher is my placeholder
toptagsreq=requests.get(f'{url}?method=artist.gettoptags&artist={artist}&api_key={YOUR_API_KEY}&format=json')
# gets the top tag for an artist specified in the F STRING {ARTIST} blob, if i change the artist in the line at the beginning it will change
toptagget=toptagsreq.json()
#returns our JSON for the request of the artist tags, I removed the print from this final submission, but it is in assignment 1
chartrequest=requests.get(f'{url}?method=tag.getweeklychartlist&tag=rock&api_key={YOUR_API_KEY}&format=json')
charts=chartrequest.json()
#toptagget gets changed in the loop that gets the tags, I'm just calling it here so I can reference it at a later point down the line.
```

Collect Artist Names

```
# Begins my counter for the loop to see what week we are in
listylist=[]
#creates an empty list for the artists to go in later
for i in charts['weeklychartlist']['chart']:
    #Begins putting us through the large loop of getting weekly data
    fromloop = charts['weeklychartlist']['chart'][chartnum]['from']
    #sets beginning of date search
    toloop = charts['weeklychartlist']['chart'][chartnum]['to']
    #sets end of date search
    weeklisteningdata = requests.get(f'{url}?method=user.getweeklyartistchart&user=rj&api_key={YOUR_API_KEY}
    #gets my data for the URL that I'm looking at
    rjdata=weeklisteningdata.json()
    #this gets the json data that I can parse through and find the lists of artists for every week
    for j in range(0, min(250, len(rjdata['weeklyartistchart']['artist']))):
        #sets my criteria that for a range between 0 and either 250 or the length of the data (whichever is
        listylist.append(rjdata['weeklyartistchart']['artist'][j]['name'])
        #Puts all of the artists into the master list, titled 'listylist'
        #print(rjdata['weeklyartistchart']['artist'][j]['name']) #this line isn't necessary, just lets me se
    chartnum+=1 #advances the chart number counter 1 item, moves onto the next week
#Breaking here so if you run and something explodes you can stop the kernel at this block.
```

Collect Tags

```
for item in listylist:
#goes through each of the items(artists) in my listylist (Master list of every artist listened to)
    itemencode = up.quote(item)
    #pencode encodes the name to combat the problem discussed above. (encoding strange characters)
    if item not in sickdict:
        #checking to see if the artist we found is in the dictionary above, if yes skip to the else condition
        toptagsreq=requests.get(f'{url}?method=artist.gettoptags&artist={itemencode}&api_key={YOUR_API_KEY}&format=json')
        # gets the top tag for an artist specified in the F STRING {itemencode} blob
        toptagget=toptagsreq.json()
        #here both toptagget and toptagsreq get reassigned from before (This is what I meant before)
        tags = toptagget['toptags']['tag']
        #gets all of the tags for a given artist {itemencode} blob above
        tag = tags[0]['name'] if len(tags) > 0 else 'unknown'
        # returns the top tag for the artist, returns unknown if no tags exist
        sickdict[item]= tag
        # puts item in the dictionary as a key, with the tag variable as the value
        #print(item,tag,counter) # Print statement again, just disabling this line so that you know it's there but it doesnt run
    else:
        pass
    #these lines make it so that i'm just moving on in the case that item is already in the dictionary, doesnt waste time
    counter+=1
    #advances the counter by 1 to move onto the next artist.
    time.sleep(0.3)
    #AI SUGGESTION, this is just so that the API doesn't take too many requests and give me a JSONDecodeError,
```

Genres Sorted

Create DataFrame

Dataframe of Artists Paired up with their raw tag

Search for keywords

Search for certain keywords to make the macro-genres

Pair Artists with Genre

This Pairs each of the Artists with a specific genre I used to calculate play share

Genres Sorted

```
#_____________________________________________________________________________________________________
#BLOCK 1.4 - Classify Genres
#As per the Code you gave me, instead of running a loop, i used np.where statements after converting my sick dictionary to a dataframe
#and used pandas to change all of the tags
sickdf=pd.DataFrame(list(sickdict.items()),columns=['artist','tags'])
#converts the dictionary to a dataframe
sickdf['tags']=sickdf['tags'].str.lower()
#converts all of the items in the tag column into lowercase strings which i use to search
sickdf['genre'] = np.where(sickdf['tags'].str.contains('pop'), 'Pop', 'other')
# creates a new column in sickdf titled 'genre' and also checks if the item contains pop, if so it will rename the item to pop, if not
#initializes the new genres to be titled 'other'
sickdf['genre'] = np.where(sickdf['tags'].str.contains('rock|punk|grunge|ska|alternative|britpop'), 'Rock', sickdf['genre'])
#checks for any of the genres in the 'rock|punk|grunge...' chunk as they are separated by or operators, if it contains any of them, it will
#rename the tag to 'Rock'. if it doesnt contain any of them it will keep the genre as other
#https://stackoverflow.com/questions/26577516/how-to-test-if-a-string-contains-one-of-the-substrings-in-a-list-in-pandas
sickdf['genre'] = np.where(sickdf['tags'].str.contains('jazz|swing|blues|soul|funk'), 'Jazz', sickdf['genre'])
# If the tag contains 'jazz|swing|blues|soul|funk' it will change the tag to Jazz if so, if not it will keep the genre the same
sickdf['genre'] = np.where(sickdf['tags'].str.contains('electronic|techno|house|ambient|trance|dubstep|trip-hop'), 'Electronic', sickdf['ge
# If the tag contains 'electronic|techno|house|ambient|trance|dubstep|trip-hop and will change the tag to Electronic if so,
#if not it will keep the genre the same
sickdf['genre'] = np.where(sickdf['tags'].str.contains('indie'), 'Indie', sickdf['genre'])
# If the tag contains 'indie' and will change the tag to Jazz if so, if not it will keep the genre the same, I know this is not necessary
# but I wanted to make sure all the formatting was homogenous
sickdf['genre'] = np.where(sickdf['tags'].str.contains('metal'), 'Metal', sickdf['genre'])
sickdf['genre'] = np.where(sickdf['tags'].str.contains('classical'), 'Classical', sickdf['genre'])
#Both metal and classical were made for the same reason as Indie above
sickdf['genre'] = np.where(sickdf['tags'].str.contains('hip|hip hop|hop|rap|Hip-Hop'), 'Hip Hop', sickdf['genre'])
# If the tag contains 'hip|hip hop|hop|rap|Hip-Hop' and will change the tag to Hip Hop if so, if not it will keep the genre the same as oth
sickdf['genre'] = np.where(sickdf['tags'].str.contains('folk|country|americana'), 'folk', sickdf['genre'])
sickdf
#returns the dataframe
#
```

New Variables Created

Play Share by Genre

% of total listening shares per week, expressed as decimal

Total Plays

Measures all plays across all artists in a given week of collection

New Variables Created

Play Share Created

```
        if tag not in genreplays:
            #if a tag is not in the dictionary, initialize it
            genreplays[tag]=0
            #including this to initialize each of the genres within the dict
        genreplays[tag]=genreplays.get(tag) + plays
        #silly refresher but i forgot how to do this https://www.digitalocean.com/community/tutorials/python-add-to-dictionary
        totalplays+=plays
        #adds the # of plays to total plays, measures how much the listener is listening per week
        #print(musician['name'],tag, f'week{chartnumber}') # Another print statement so i disabled,prints the musician, their tag, and the ch

    # essentially this for loop is putting ALL of my stuff into the empty list so that I can put it into a dataframe
    for genre, plays in genreplays.items():
        #gets both genre and playshare from my dictionary (pretty much key and vlaue)
        dataframe.append({'Week':fromdate,'Genre':genre, 'Play Share': plays/totalplays,'Total Plays':totalplays})
        #appends all of these different pairs into my datafram with the key as the Title of the column and the
        #fromdate is in a disgusting format, gonna look this up to change it, https://stackoverflow.com/questions/17594298/date-time-formats-
    chartnumber+=1
    #advances the chart number up by 1
    time.sleep(0.3)
    #pauses the for loop so that I don't overload my API
#
```

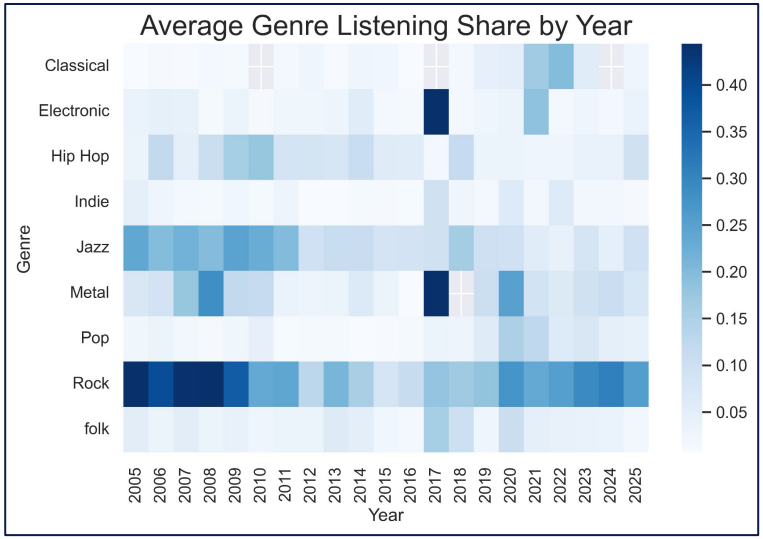
New Variables Created

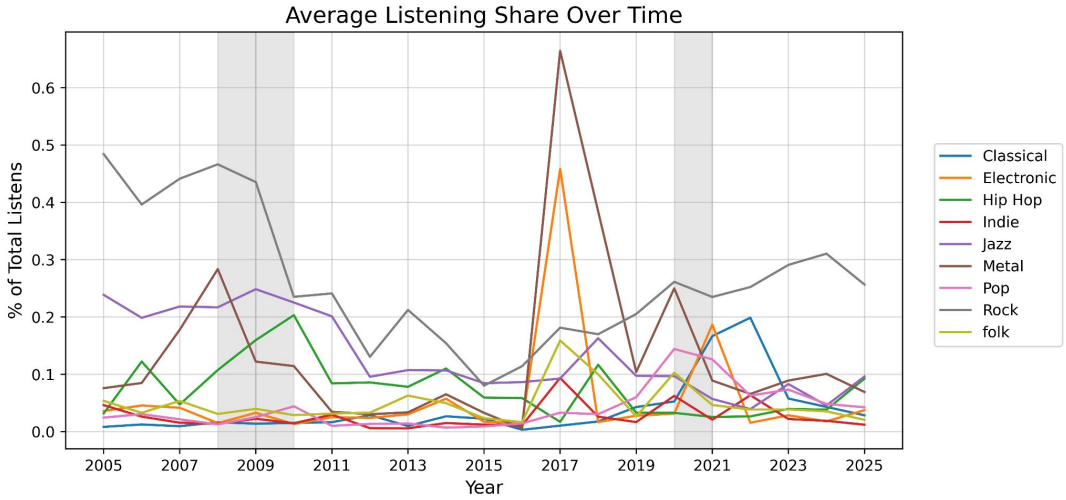
Total Plays Initialized

Adds each artist's plays to Total Plays

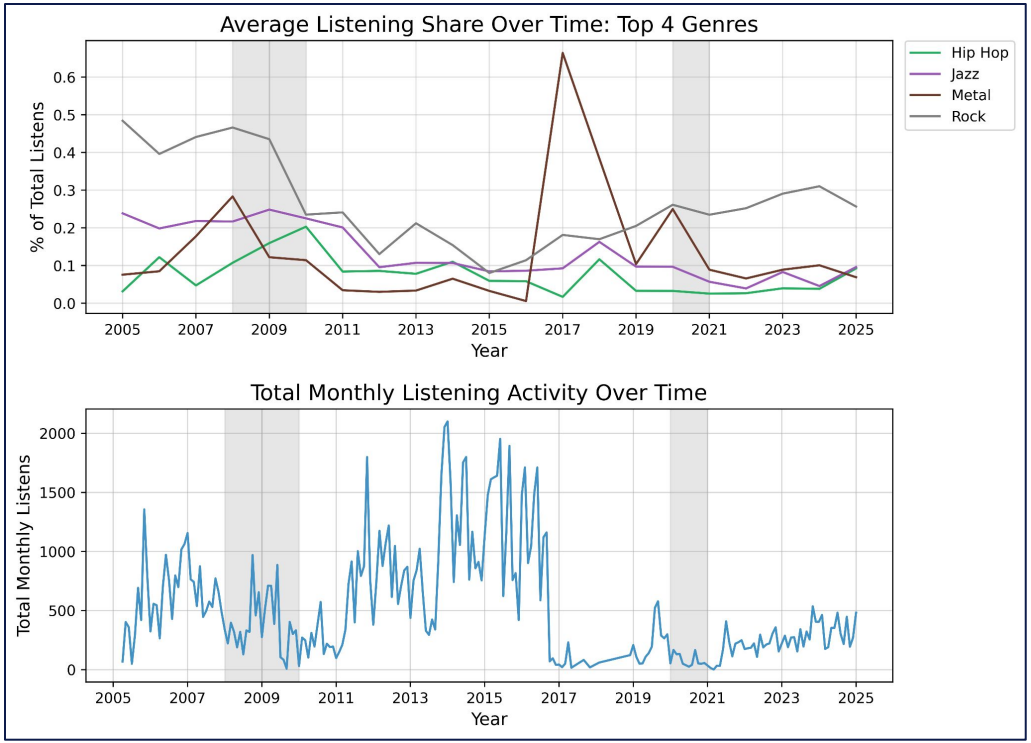
```
for chart in charts['weeklychartlist']['chart']:
    fromdate = charts['weeklychartlist']['chart'][chartnumber]['from']
    #sets date to begin at for the weeklisteningdata search
    todate = charts['weeklychartlist']['chart'][chartnumber]['to']
    #sets end date for the weeklisteningdata
    weeklisteningdata = requests.get(f'{url}?method=user.getweeklyartistchart&user=rj&api_key={YOUR_API_KEY}&format=json&to={todate}&from={fr
    #the above line gets the weekly artist chart for the week specified by the fromdate and todate variables
    weekjson=weeklisteningdata.json()
    #gets the above request as a JSON print
    weekartists=weekjson['weeklyartistchart']['artist'][:250]
    #gets first 250 artists, representative of most of the listening for the week
    #essentially this next for loop tracks each weeks total genreplays and total plays in general
    genreplays={}
    #creates an empty dictionary for every week in the loop, use this to see genre shares for each of the weeks
    totalplays=0
    #initializes the total plays so we can see out of the total plays per week, how many were each genre
    for musician in weekartists:
        #cycles through each of the top 250 artists per week
        name=musician['name']
        #gets artist's name
        plays=int(musician['playcount'])
        #how many times they were played during this week, gets added to total plays and then calculated as a share of listening
        tag=sickdict.get(name,'other')
        #gets the name of the artist from the sick dictionary returns the genre as a tag
        if tag not in genreplays:
            #if a tag is not in the dictionary, initialize it
            genreplays[tag]=0
            #including this to initialize each of the genres within the dict
        genreplays[tag]=genreplays.get(tag) + plays
        #silly refresher but i forgot how to do this https://www.digitalocean.com/community/tutorials/python-add-to-dictionary
        totalplays+=plays
        #adds the # of plays to total plays, measures how much the listener is listening per week
        #print(musician['name'],tag, f'week{chartnumber}') # Another print statement so i disabled,prints the musician, their tag, and the ch
```

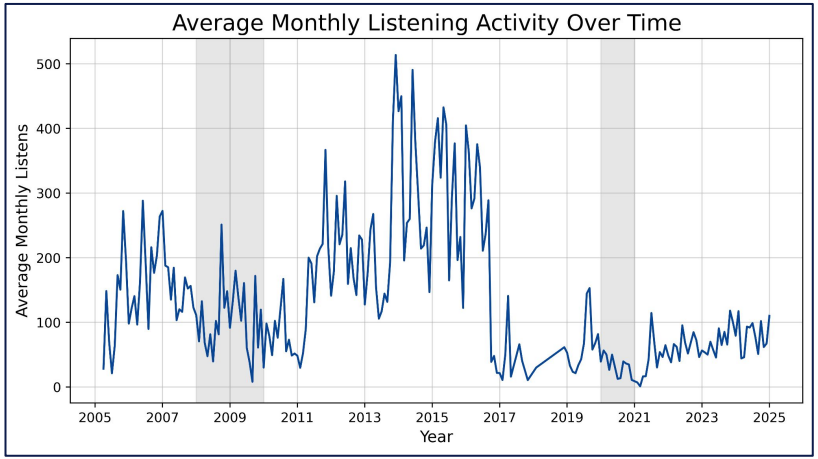
Average Listening Shares by Genre





Filtered Graphs





Models

Total Month Plays:

$$Total.Month.Plays = \beta 0 + \beta 1(REC) + \beta 2(UNRATE) + \beta 3\ln(PCE) + \beta 4\ln(UMICH) + \varepsilon$$

Logged Total Month Plays:

$$\ln(Total.Month.Plays) = \beta 0 + \beta 1(REC) + \beta 2(UNRATE) + \beta 3\ln(PCE) + \beta 4\ln(UMICH) + \varepsilon$$

Logged Genre Play Share:

$$\ln(Play.Share_i) = \beta 0 + \beta 1(REC) + \beta 2(UNRATE) + \beta 3\ln(Total.Plays) + \beta 4\ln(PCE) + \beta 5\ln(UMICH) + \varepsilon$$

**i is an individual genre*

Total Plays Regression

Recession's Impact on Total Plays

	Total Plays	
	Total Play Change	% Change
	(1)	(2)
REC	-234.352*	-0.595**
	(120.191)	(0.295)
UNRATE	-1.713	-0.105**
	(17.241)	(0.042)
log(PCE)	-1,067.683***	-3.879***
	(331.641)	(0.815)
log(UMICH)	65.854	-1.574***
	(223.498)	(0.549)
Constant	5,183.800**	31.133***
	(2,296.576)	(5.645)
Observations	232	232
R^2	0.088	0.091
Adjusted R^2	0.072	0.075
Residual Std. Error (df = 227)	447.301	1.100
F Statistic (df = 4; 227)	5.479***	5.675***
Note:	*p<0.1; **p<0.05; ***p<0.01	

Findings

- Recessions **do** cause a decrease in listening (nearly 60%)
- Unemployment has little impact in play change, larger in % change
- Inflation has a very dramatic effect on listening habits
- UMich Survey is only significant when it is logged

Genre Regressions: Top 4

Recession's Impact on Playshare by Genre: Top 4 Genres

	Logged Play Share			
	Rock	Metal	Jazz	Hip Hop
	(1)	(2)	(3)	(4)
REC	-0.080	0.722***	-0.211	0.099
	(0.158)	(0.230)	(0.165)	(0.275)
UNRATE	-0.189***	-0.178***	0.026	0.108***
	(0.024)	(0.035)	(0.025)	(0.034)
log(Total.Plays)	-0.256***	-0.748***	-0.182***	-0.392***
	(0.037)	(0.067)	(0.043)	(0.064)
log(PCE)	-4.465***	-3.337***	-4.995***	-0.254
	(0.440)	(0.657)	(0.461)	(0.642)
log(UMICH)	-2.614***	-0.653	-0.834**	0.352
	(0.309)	(0.558)	(0.337)	(0.475)
Constant	32.342***	19.452***	24.741***	-2.307
	(3.165)	(5.014)	(3.311)	(4.681)
Observations	788	409	625	488
R^2	0.198	0.334	0.252	0.105
Adjusted R^2	0.193	0.326	0.246	0.095
Residual Std. Error	1.039 (df = 782)	1.153 (df = 403)	0.965 (df = 619)	1.153 (df = 482)
F Statistic	38.711*** (df = 5; 782)	40.455*** (df = 5; 403)	41.641*** (df = 5; 619)	11.259*** (df = 5; 482)
Note:				*p<0.1; **p<0.05; ***p<0.01

- Metal experience an **increase** in play share during recessions, Hip hop is + but not significant
- 1 PP increase in Unemployment decreases Rock and Metal, increases Jazz and Hip Hop
- As total plays increase, genres get more diverse
- Inflation again is very dramatically negative, except for Hip Hop
- UMich decreases all significantly, except for Hip Hop

Genre Regressions: Special Cases

Recession's Impact on Playshare by Genre: Special Cases

	Logged Play Share	
	Pop	Classical
	(1)	(2)
REC	-0.128	0.006
	(0.199)	(0.527)
UNRATE	-0.081***	-0.085
	(0.025)	(0.080)
log(Total.Plays)	-0.897***	-0.647***
	(0.048)	(0.139)
log(PCE)	0.950**	3.492**
	(0.452)	(1.320)
log(UMICH)	0.426	-0.667
	(0.309)	(1.004)
Constant	-5.409*	-13.444
	(3.229)	(9.560)
Observations	393	75
R^2	0.561	0.528
Adjusted R^2	0.555	0.494
Residual Std. Error	0.775 (df = 387)	0.921 (df = 69)
F Statistic	98.930*** (df = 5; 387)	15.429*** (df = 5; 69)
Note:		*p<0.1; **p<0.05; ***p<0.01

- Recession Pop does **not** exist for these models (no statistical significance)
- Unemployment Rate slightly affects Pop's Share
- Total Plays follows the same trend as other genres
- Inflation increases playshare
- More pop when sentiment increases, less classical (no statistical significance)

Future Improvements

- **Use actual global data not one person**
- **Take into account song content**
  - Control for mood (happy vs sad rock)
  - Control for keywords
- **Find different method to get genres**
  - User generated tags, not homogenous
- **Use Fixed Effects to control for more popular genres**
  - ‘Classic Rock’ is going to have a different listen count than PBR&B or Shoegaze, larger more macro genre

Thank you!